Chuleta y Macro-Resumen de Examen

Despliegue de Aplicaciones Web (2º DAW) · Git, GitHub Pages y GitHub Actions

DIRECTO AL GRANO | EJEMPLOS COPIABLES | WORKFLOWS COMPLETOS | EXAMEN RESUELTO

1 Las 5 Reglas Sagradas y Configuración Inicial

Las 5 Reglas del Examen:

1. **Antes de cada paso:** Ejecuta `git status` y `git branch` (¿dónde estoy y qué tengo pendiente?).
2. **Sintaxis exacta:** Todo comando empieza por `git`. Los flags van **sin espacio** (`--soft` , `--hard` , `--staged`).
3. **Cambios reales:** Para generar cambios usa `echo "algo" >> archivo` o `nano`. Un comando `touch` en un archivo ya existente no cambia nada.
4. **Push de tags:** Un `git push` normal **NO** sube tags. Se suben aparte: `git push origin <tag>` .
5. **Comprobación final:** Verifica siempre el árbol con `git log --oneline --graph --all --decorate` .

Configuración de Identidad en Terminal

```
# Configurar nombre y correo
git config --global user.name "TuNombre Apellido"
git config --global user.email "tu@email.com"

# Evitar que se abra Vim
git config --global core.editor nano

# Guardar credenciales para no pedir token cada vez
git config --global credential.helper store

# Comprobar configuración guardada
git config --list
```

Personal Access Token (PAT) de GitHub

GitHub no admite tu contraseña normal en la terminal. El token es obligatorio:

- **Crear:** GitHub → Avatar → *Settings* → *Developer settings* → *Personal access tokens* → *Tokens (classic)* → *Generate*.
- **Permisos indispensables:** Marcar `repo` (completo) y `workflow` .
- **En el git push:** `Username:` tu usuario | `Password:` pega el token (no salen letras ni asteriscos).

2 Terminal de Rescate y Git Local (Las 3 Zonas)

Atajos y Comandos de Terminal

```
# Navegación y carpetas
pwd && ls -la                 # Dónde estoy y ver todo
mkdir -p a/b/c                # Crear árbol de carpetas
chmod +x script.sh            # Hacer ejecutable (vital e
n Actions)

# Escribir varias líneas rápido (Heredoc)
cat > index.html << 'EOF'
<!DOCTYPE html>
<html><body><h1>Web Base</h1></body></html>
EOF

# Salir de editores si te atrapan
Nano: Ctrl+O, Enter (guardar) -> Ctrl+X (salir)
Vim:  Esc -> :wq Enter (guardar) | :q! Enter (descartar)
Log bloqueado: Pulsa la tecla 'q'
```

Ciclo de Trabajo y Zonas

```
# 1. Iniciar repo
git init

# 2. Comprobar estado
git status

# 3. Preparar cambios (Staging)
git add archivo.html          # Solo uno
git add .                     # Todo

# 4. Confirmar (Repositorio)
git commit -m "Mensaje imperativo"

# Atajo add + commit (solo archivos ya rastreados)
git commit -am "Actualiza cambios"

# Comparar
git diff                      # Working vs Staging
git diff --staged             # Staging vs Último commit
```

Gestión de .gitignore

```
# Ejemplo de .gitignore
*.log
node_modules/
dist/
.env
venv/
!importante.log  # Excepción (se sube)
```

Si ya hiciste commit de un fichero ignorado:
`git rm --cached archivo.log && git commit -m "Ignora log"`

Deshacer Cambios y Errores

```
# Sacar de staging a working:
git restore --staged archivo.txt

# Descartar cambios de un archivo modificado:
git restore archivo.txt

# Cambiar mensaje o sumar archivos al último commit:
git commit --amend -m "Nuevo mensaje" --no-edit

# Deshacer commits (HEAD~1 = último commit):
git reset --soft HEAD~1   # Conserva cambios en Staging
git reset HEAD~1          # (Mixed) Conserva cambios en Working
git reset --hard HEAD~1   # BORRA commit y cambios del disco
git revert <HASH>         # Nuevo commit que anula el anterior
```

3 Ramas, Merge, Conflictos y Tags

Ramas y Fusión (Práctica de Examen)

```
# Listar y ver rama activa
git branch
git branch --show-current

# Renombrar rama actual (ej: main a master o al revés)
git branch -m master

# Crear rama y entrar en ella
git switch -c contacte        # o: git checkout -b contac
te

# Trabajar en rama contacte
cat > index.html << 'EOF'
<h1>Contacto</h1>
EOF
git add index.html && git commit -m "Añade contacto"

# Volver a master y hacer otro commit (DIVERGENCIA)
git switch master
echo '<meta name="desc" content="web">' > new.html
git add new.html && git commit -m "Añade new.html"

# Fusionar contacte en master (crea commit de 3 vías)
git merge contacte

# Borrar rama ya fusionada
git branch -d contacte        # -D si se fuerza borrado
```

Resolución de Conflictos

Si dos ramas modificaron las mismas líneas, salta `CONFLICT (content)`:

```
<<<<<<< HEAD
<h1>Título en rama master</h1>
=======
<h1>Título en rama contacte</h1>
>>>>>>> contacte
```

```
# 1. Editar el archivo con nano y borrar las marcas
# 2. Guardar el archivo limpio
# 3. Completar el merge:
git add index.html
git commit -m "Resuelve conflicto de cabecera"

# O si quieres cancelar el merge entero:
git merge --abort
```

Tags (Etiquetas de Entrega)

```
# Crear tag anotado (autor, fecha y mensaje)
git tag -a entrega-tunombre -m "Entrega de la prueba"

# Ver contenido del tag
git show entrega-tunombre

# SUBIR TAGS (Un push normal NO los sube)
git push origin entrega-tunombre    # Este tag concreto
git push origin --tags              # Todos los tags

# Borrar tag:
git tag -d v1.0                     # Local
git push origin --delete v1.0       # En GitHub
```

4 Remotos en GitHub, Fork, PR y GitHub Pages

Remotos y Flujo Fork + Upstream + PR

```
# 1. Vincular repo local con GitHub
git remote add origin https://github.com/USUARIO/REPO.git
git push -u origin master           # Solo la primera vez
git push                            # Siguientes veces

# Corregir URL remota si te equivocaste (NO uses add otra
vez):
git remote set-url origin https://github.com/USUARIO/CORR
ECTO.git

# Sincronizar Fork con el repo del profesor (upstream):
git remote add upstream https://github.com/PROFE/ORIGINA
L.git
git fetch upstream
git merge upstream/main

# Crear PR:
git switch -c mejora
git commit -am "Mejora" && git push -u origin mejora
# Web GitHub -> Pull Requests -> New -> base: original, c
ompare: mejora
```

GitHub Pages (Poner la Web Online)

> **Web de Usuario:** El repo debe llamarse exactamente `TUUSUARIO.github.io`.
> Archivos (`index.html` , `css/` , `js/`) directamente en la raíz (no en subcarpetas).

```
# Subir contenido web:
mkdir ~/web && cd ~/web
unzip web.zip                       # Comprobar que inde
x.html queda aquí
git init && git add . && git commit -m "Web inicial"
git remote add origin https://github.com/USUARIO/USUARIO.
github.io.git
git push -u origin master           # o main

# Activar en GitHub:
Repo -> Settings -> Pages -> Source: Deploy from a branch
Seleccionar rama: master (o main) / (root) -> Save.
URL: https://USUARIO.github.io/ (esperar 1-2 min, Ctrl+F
5).
```

5 Teoría SCV y Metodología Gitflow

Tipo SCV	Arquitectura	Ventajas / Inconvenientes	Ejemplos
Local	Base de datos de revisiones en tu propio disco duro.	Sencillo; sin colaboración y punto único de pérdida.	RCS
Centralizado	Un servidor único tiene el historial; clientes bajan solo la versión de trabajo.	Fácil control; si el servidor cae nadie puede hacer commits ni trabajar.	CVS, SVN
Distribuido	Cada cliente tiene un clon íntegro del repositorio e historial.	Trabajo offline, copias de seguridad automáticas, tolerancia a fallos.	Git, Mercurial

Ramas en Gitflow

- `master` / `main` : Producción estable. Vive siempre.
- `develop` : Próxima versión. Vive siempre. Nace de master.
- `feature/*` : Funcionalidades nuevas. Nace de develop, fusiona en develop.
- `release/*` : Preparación de versión y fixes menores. Nace de develop, fusiona en master (+tag) y develop.
- `hotfix/*` : Parche urgente en producción. Nace de master, fusiona en master (+tag) y develop.

Comandos Gitflow vs Git Estándar

```
# Iniciar Gitflow:
git flow init                          # o: git checkout
-b develop master

# Feature:
git flow feature start login           # git switch -c f
eature/login develop
git flow feature finish login          # switch develop
-> merge --no-ff -> branch -d

# Release:
git flow release start 1.0.0           # git switch -c r
elease/1.0.0 develop
git flow release finish 1.0.0          # merge en master
con tag y en develop
```

6 MACRO-SECCIÓN: GitHub Actions (Workflows y Sintaxis)

> **¡Fallo típico n.º 1 en examen!** La ruta del archivo YAML debe ser `.github/workflows/nombre.yml` (con **'s' final en workflows**). Si falta la s o la carpeta está mal llamada, GitHub no dará ningún error pero Actions no ejecutará nada.

Palabras Clave de un Workflow

Clave	Significado y Uso
`name`	Nombre identificativo del workflow, job o step.
`on`	**Evento disparador** (push, pull_request, cron).
`jobs`	Conjunto de trabajos. Por defecto corren en **paralelo**.
`runs-on`	Máquina runner (`ubuntu-latest` , `windows-latest` , `macos-latest`).
`steps`	Pasos secuenciales del job (empiezan con guion `-`).
`uses`	Ejecuta una Action reutilizable (ej: `actions/checkout@v4`).
`with`	Parámetros que se pasan a la action (ej: `python-version: '3.12'`).
`run`	Ejecuta un comando de terminal del sistema.
`needs`	Espera a que otro job termine con éxito (ej: `needs: build-linux`).
`if`	Condicional (ej: `if: github.ref_name == 'main'`).
`env`	Variables de entorno (a nivel workflow, job o step).

Disparadores (`on`), Filtros y Cron

```yaml
name: Triggers y Filtros
on:
  push:
    branches:
      - main
      - 'releases/**'       # Coincidencia con comodín
      - '!releases/**-alpha'# Exclusión con !
    paths:
      - 'src/**'            # Solo si cambian ficheros de src/
      - 'requirements.txt'
    tags:
      - 'v*'                # Solo al subir tags como v1.0
  pull_request:
    branches: [ main ]
  workflow_dispatch:        # Botón manual en la web de GitHub
  schedule:
    - cron: '30 7 * * 1'    # Lunes a las 07:30 UTC
    # Sintaxis cron: min(0-59) hora(0-23) día(1-31) mes(1-12) díaSemana(0-6, 0=Dom)
```

Contextos y Secretos

```yaml
# Expresiones útiles:
${{ github.ref_name }}       # Rama o tag que disparó la acción
${{ github.sha }}            # Hash SHA del commit
${{ github.actor }}          # Usuario que hizo la acción
${{ runner.os }}             # Linux, Windows o macOS
${{ secrets.MI_TOKEN }}      # Secreto guardado en Repo -> Settings -> Secrets
${{ steps.id_paso.outputs.x }}# Output devuelto por un step previo
```

Artefactos entre Jobs y Estrategia Matricial

¿Por qué hacen falta artefactos y `needs` ?

Cada job se ejecuta en una máquina virtual limpia e independiente. Los ficheros creados en el Job A **se destruyen** al acabar. Para pasarlos al Job B:

1. Job A usa `actions/upload-artifact@v4` con un `name` y `path` .
2. Job B declara `needs: JobA` para esperarlo.
3. Job B descarga el archivo con `actions/download-artifact@v4` usando el mismo `name` .

Estrategia Matricial (`strategy.matrix`)

```yaml
jobs:
  test-matrix:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false       # No cancela las demás si una falla
      matrix:
        os: [ ubuntu-latest, windows-latest ]
        python-version: [ '3.11', '3.12' ]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: python --version
```

7 Plantilla Examen 1: Multi-SO y Artefactos (PyInstaller)

Este es el ejercicio clásico completo: compila un script binario en Linux, macOS y Windows, y en un job posterior descarga el binario Linux, le otorga permisos `chmod +x` y lo ejecuta.

```yaml
# Fichero: .github/workflows/empaquetar.yml
name: Empaquetat

on:
  push:
    branches:
      - master                # OJO: Pon el nombre real de tu rama (master o main)

jobs:
  build-linux:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout del repositorio
        uses: actions/checkout@v4

      - name: Configurar Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Instalar paquete pyinstaller
        run: pip install pyinstaller

      - name: Empaquetar script con pyinstaller
        run: pyinstaller --onefile comprova_desplegaments.py

      - name: Guardar ejecutable como artefacto
        uses: actions/upload-artifact@v4
        with:
          name: executable-linux
          path: dist/comprova_desplegaments

  build-macos:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install pyinstaller
      - run: pyinstaller --onefile comprova_desplegaments.py

  build-windows:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install pyinstaller
      - run: pyinstaller --onefile comprova_desplegaments.py

  comprova:
    needs: build-linux         # Espera que termine build-linux
    runs-on: ubuntu-latest
    steps:
      - name: Descargar ejecutable de Linux
        uses: actions/download-artifact@v4
        with:
          name: executable-linux

      - name: Dar permisos de ejecucion
        run: chmod +x comprova_desplegaments

      - name: Ejecutar la comprobacion
        run: ./comprova_desplegaments example.com
```

8 Plantilla Examen 2: Modificaciones euro.py y Versión Pro

Las 5 Tareas Pedidas en la Práctica

- **1 & 2. Clonar y cambiar URL remota:**
  `git clone https://github.com/titansmc/actions.git && cd actions`
  `git remote set-url origin https://github.com/USUARIO/actions.git`
  `git push -u origin main`
- **3. Paquetes desde requirements.txt:**
  Crear `requirements.txt` con `requests` y `pandas`.
  En el YAML: `run: pip install -r requirements.txt`
- **4. Rama test con Python 3.12:**
  `git switch -c test` → Cambiar en YAML `python-version: '3.12'` y añadir `test` a branches → `git push -u origin test`
- **5. Rama development solo con USD:**
  `git switch main && git switch -c development` → Cambiar en YAML: `run: python ./euro.py USD` y añadir rama a branches → `git push -u origin development`

Versión Pro Unificada (1 Solo YAML para las 3 ramas)

```yaml
name: Monedas-Universal
on:
  push:
    branches: [ main, test, development ]
jobs:
  create-list:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          # Operador ternario según la rama actual:
          python-version: ${{ github.ref_name == 'test' &
& '3.12' || '3.11' }}
      - run: pip install -r requirements.txt

      # Si es main o test: corre normal
      - if: github.ref_name != 'development'
        run: python ./euro.py

      # Si es development: pasa el argumento USD
      - if: github.ref_name == 'development'
        run: python ./euro.py USD
```

9 Custom Actions (Creación y Uso)

Estructura y Tipos de Custom Action

- **Docker Action:** Solo corre en runners Linux. Lenta en compilar pero garantiza entorno cerrado.
- **JavaScript Action:** Multiplataforma (Linux, Mac, Windows) y rápida. Requiere Node.
- **Composite Action:** Encadena varios steps directos de terminal en una sola acción.

Campos obligatorios en action.yml: `name`, `description`, `runs`.

Cómo Invocar la Custom Action en un Workflow

```yaml
jobs:
  probar-custom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4   # OBLIGATORIO para tener los archivos
      - id: saluda
        uses: ./mi-accion           # Carpeta donde está el action.yml
        with:
          who-to-greet: 'Valen'
      - run: echo "Hora: ${{ steps.saluda.outputs.time }}"
```

Ejemplo Completo: Docker Custom Action

1. action.yml:

```yaml
name: 'Hola Mundo Docker'
description: 'Saluda y devuelve la hora'
inputs:
  who-to-greet:
    description: 'Nombre'
    required: true
    default: 'Mundo'
outputs:
  time:
    description: 'Hora actual'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

2. Dockerfile:

```
FROM alpine:3.18
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

3. entrypoint.sh:

```
#!/bin/sh -l
echo "Hola $1"
# Sintaxis moderna de outputs en Actions:
echo "time=$(date)" >> $GITHUB_OUTPUT
```

10 Tabla Rápida: «Si el enunciado pide... tú ejecutas...»

Lo que pide el enunciado	Comando o Sintaxis Exacta
Poner nombre y correo	`git config --global user.name "..." && git config --global user.email "..."`
Crear repositorio local	`mkdir carpeta && cd carpeta && git init`
Ver en qué rama estás	`git branch` o `git branch --show-current`
Renombrar rama principal	`git branch -m master` (o `main`)
Crear rama y entrar en ella	`git switch -c nombre-rama` (o `git checkout -b nombre-rama`)
Deshacer último commit sin perder archivos	`git reset --soft HEAD~1`
Fusionar rama en la actual	`git merge nombre-rama`
Crear tag anotado de entrega	`git tag -a entrega-tunombre -m "Entrega examen"`
Subir cambios de la rama por 1ª vez	`git push -u origin <nombre-rama>`
Subir el tag al repositorio remoto	`git push origin entrega-tunombre` (¡el push normal no lo sube!)
Cambiar la URL del remoto equivocada	`git remote set-url origin https://github.com/USUARIO/REPO.git`
Workflow: solo en ramas específicas	`on: push: branches: [ main, develop ]`
Workflow: descargar código al runner	`- uses: actions/checkout@v4`
Workflow: configurar versión Python	`- uses: actions/setup-python@v5 with: python-version: '3.12'`
Workflow: instalar dependencias de archivo	`- run: pip install -r requirements.txt`
Workflow: compartir ejecutable con otro job	`upload-artifact@v4` en el primero, `needs:` y `download-artifact@v4` en el segundo

11 Errores Típicos en Examen y Solución

Error o Síntoma	Causa Real	Solución Inmediata
`fatal: not a git repository`	No estás dentro de la carpeta del proyecto.	Haz `cd` a la carpeta o ejecuta `git init`.
`fatal: remote origin already exists`	Ya ejecutaste `git remote add` antes.	Usa `git remote set-url origin <URL>` para modificarla.
`error: src refspec master does not match any`	Tu rama local se llama `main` o no has hecho ningún commit aún.	Haz el commit inicial o renombra con `git branch -m master`.
`fatal: Authentication failed`	Pusiste tu contraseña habitual de GitHub.	Usa el **Personal Access Token (classic)** como password.
`refusing to allow PAT to create/update workflow`	El token creado no tiene el permiso de workflows.	Genera un token nuevo en GitHub marcando la casilla **`workflow`**.
`[rejected - non-fast-forward]`	El repo en GitHub se creó con README y divergió del tuyo.	Crea el repo en GitHub **vacío**, o haz `git pull origin master --allow-unrelated-histories`.
En la pestaña Actions no sale ninguna ejecución	Carpeta mal llamada o rama no coincidente.	Verifica que la carpeta sea **`.github/workflows/`** **(con 's' final)** y que la rama del push esté en `branches:`.
El step de Action dice `file not found`	No se clonó el código dentro de la máquina runner.	Añade `- uses: actions/checkout@v4` como primer paso del job.
`Permission denied` al correr un binario en Linux	Faltan permisos de ejecución.	Añade `run: chmod +x ./archivo` antes de llamarlo.
Los tags no se ven en GitHub	Hiciste `git push` sin especificar tags.	Ejecuta `git push origin <nombre-tag>` o `git push origin --tags`.

12 Checklist Final de 1 Minuto antes de Entregar

- [ ] **1. git status limpio:** No quedan ficheros en rojo sin añadir ni cambios sin commitear.
- [ ] **2. En la rama requerida:** Ejecuta `git branch` y confirma que estás en la rama que pide el profesor (master, main...).
- [ ] **3. Histórico verificado:** Ejecuta `git log --oneline --graph --all --decorate` y revisa que las fusiones y tags están en su sitio.
- [ ] **4. Ramas secundarias:** Si el enunciado pedía borrar las ramas temporales tras el merge, confirma con `git branch` que ya no existen.
- [ ] **5. Tags en GitHub:** Entra en GitHub → Pestaña **Tags** y comprueba que aparece tu tag de entrega.
- [ ] **6. Actions en verde:** Entra en GitHub → Pestaña **Actions**. La ejecución de tu último commit debe tener el icono **verde** (todos los jobs pasados). Si está en rojo, entra al job y lee el paso fallido.
- [ ] **7. Si el ejercicio es GitHub Pages:** Abre `https://TUUSUARIO.github.io/` en incógnito o pulsa `Ctrl + F5` para confirmar que se carga la web.